Day 10: Lists and Tuples

1. Creating Lists and Tuples

Until now each variable held one value. **Collections** hold many values under one name. Today covers the two most common sequence types: lists and tuples. Security scripts use them constantly, for example a list of ports to check, usernames to audit, or lines read from a log.

Lists

A **list** is an ordered, **changeable** (mutable) collection written with square brackets.

python

```
ports = [22, 80, 443]
services = ["ssh", "http", "https"]
mixed = ["admin", 3, True, 2.5]      # types can be mixed
empty = []                           # an empty list
```

Useful basics:

python

```
print(ports)            # [22, 80, 443]
print(type(ports))      # <class 'list'>
print(len(ports))       # 3
print(80 in ports)      # True
print(25 not in ports)  # True
```

Creating lists from other things:

python

```
letters = list("SSH")             # ['S', 'S', 'H']
numbers = list(range(1, 6))       # [1, 2, 3, 4, 5]
words = "ssh http https".split()  # ['ssh', 'http', 'https']
```

Joining and repeating lists:

python

```
print([1, 2] + [3, 4])    # [1, 2, 3, 4]
print([0] * 3)            # [0, 0, 0]
```

Lists can hold lists (a table-like structure):

python

```
hosts = [["web", 80], ["ssh", 22]]
```

print(hosts[0][1]) # 80

Tuples

A **tuple** is an ordered, **unchangeable** (immutable) collection written with round brackets.

python

server = ("192.168.56.101", 22)

days = ("Mon", "Tue", "Wed")

One-item tuples need a trailing comma, or Python sees only brackets:

python

single = (5,) # tuple

not_tuple = (5) # just the integer 5

Brackets are optional (point = 3, 4 also makes a tuple), but they make code clearer.

Unpacking a tuple into variables is very handy:

python

host, port = ("192.168.56.101", 22)

print(host) # 192.168.56.101

print(port) # 22

List vs. tuple

Feature	**List**	**Tuple**
Brackets	[ ]	( )
Can change items	Yes	No
Add or remove items	Yes	No
Typical use	Data that grows or changes	Fixed data (coordinates, settings, a host and port pair)
Speed and memory	Slightly heavier	Slightly lighter

Why use a tuple? It signals "this should not change" and protects data from accidental edits. Example: a fixed set of allowed protocols.

python

ports = (80, 443)

ports[0] = 8080 # TypeError: 'tuple' object does not support item assignment

Converting between them:

python

t = tuple([1, 2, 3]) # (1, 2, 3)

l = list((1, 2, 3)) # [1, 2, 3]

To “change” a tuple, convert it to a list, edit, and convert back.

2. Indexing and Slicing

Indexing and slicing work **exactly like strings** (Day 6): index starts at 0, negative indexes count from the end, and the slice stop is excluded.

```
 "ssh"  "http"  "https"  "ftp"
   0      1       2       3       (positive)
  -4     -3      -2      -1       (negative)
```

Indexing

python

```
services = ["ssh", "http", "https", "ftp"]


print(services[0])   # ssh
print(services[2])   # https
print(services[-1])  # ftp
print(services[-2])  # https
```

python

```
print(services[10])  # IndexError: list index out of range
```

Changing an item (lists only):

python

```
services[1] = "HTTP"
print(services)      # ['ssh', 'HTTP', 'https', 'ftp']
```

Slicing: [start:stop:step]

python

```
ports = [21, 22, 23, 25, 53, 80, 110, 443]


print(ports[0:3])    # [21, 22, 23]
print(ports[:3])     # [21, 22, 23]      first three
print(ports[5:])     # [80, 110, 443]    from index 5 on
print(ports[-3:])    # [80, 110, 443]    last three
```

print(ports[::2]) # [21, 23, 53, 110] every second item

print(ports[::-1]) # [443, 110, 80, 53, 25, 23, 22, 21] reversed

Out-of-range slices don't crash; they return what exists:

python

print(ports[0:100]) # the whole list

Slicing makes a copy:

python

backup = ports[:] # a new list with the same items

Replacing a section (lists only):

python

nums = [1, 2, 3, 4, 5]

nums[1:3] = [20, 30]

print(nums) # [1, 20, 30, 4, 5]

Other handy tools

python

nums = [4, 9, 1, 7]

print(len(nums)) # 4

print(max(nums)) # 9

print(min(nums)) # 1

print(sum(nums)) # 21

print(nums.index(1)) # 2 (position of the first 1; error if missing)

print(nums.count(7)) # 1 (how many times it appears)

Tuples support all of these **read-only** operations too (indexing, slicing, len, count, index, in).

3. Adding and Removing List Items

These change the list **in place**. They apply to **lists only**, not tuples.

Adding items

Method	What it does	Example
append(x)	Adds one item at the end	ports.append(8080)
insert(i, x)	Adds at position i	ports.insert(0, 21)

extend(iterable)	Adds every item of another list	ports.extend([25, 53])

python

```
ports = [22, 80]


ports.append(443)

print(ports)          # [22, 80, 443]


ports.insert(0, 21)

print(ports)          # [21, 22, 80, 443]


ports.extend([8080, 8443])

print(ports)          # [21, 22, 80, 443, 8080, 8443]
```

append vs. extend:

python

```
a = [1, 2]

a.append([3, 4])

print(a)      # [1, 2, [3, 4]]   a list inside the list


b = [1, 2]

b.extend([3, 4])

print(b)      # [1, 2, 3, 4]
```

Removing items

Method	**What it does**
remove(x)	Deletes the first item equal to x (error if missing)
pop()	Removes and **returns** the last item
pop(i)	Removes and returns the item at index i
del list[i]	Deletes the item at index i (also works on slices)
clear()	Empties the whole list

python

```
users = ["alice", "bob", "carol", "dave", "erin"]
```

```
users.remove("bob")
print(users)          # ['alice', 'carol', 'dave', 'erin']

last = users.pop()
print(last)           # erin
print(users)          # ['alice', 'carol', 'dave']

first = users.pop(0)
print(first)          # alice

del users[0]
print(users)          # ['dave']

users.clear()
print(users)          # []
```

Avoid crashes: remove() and pop() raise errors on bad input, so check first.

```
python
users = ["alice", "carol"]

if "bob" in users:
    users.remove("bob")
else:
    print("bob is not in the list")

if users:             # only pop if the list is not empty
    users.pop()
```

Sorting and reversing

```
python
ports = [443, 22, 80, 21]

ports.sort()                # changes the list itself
```

```
print(ports)                # [21, 22, 80, 443]


ports.sort(reverse=True)
print(ports)                # [443, 80, 22, 21]


ports.reverse()
print(ports)                # [21, 22, 80, 443]


original = [3, 1, 2]
print(sorted(original))     # [1, 2, 3]  (returns a new list)
print(original)             # [3, 1, 2]  (unchanged)
```

Common trap: sort(), append(), remove() and reverse() return None, not the list.

python

```
ports = [3, 1, 2]
ports = ports.sort()    # WRONG: ports is now None
```

Copying lists: a classic beginner trap

python

```
a = [1, 2, 3]
b = a           # NOT a copy: both names point to the same list
b.append(4)
print(a)        # [1, 2, 3, 4]   a changed too!


c = a.copy()    # real copy (or a[:] or list(a))
c.append(5)
print(a)        # [1, 2, 3, 4]   unaffected
```

4. Looping Through Lists

You met this on Day 8. Now lists make it powerful.

Loop over items

python

```
ports = [22, 80, 443]
```

```
for port in ports:
    print(f"Checking port {port}")
```

Loop with index and item: enumerate()

```
python
users = ["alice", "bob", "carol"]


for i, name in enumerate(users, start=1):
    print(f"{i}. {name}")
```

Loop by position with range(len())

Use this only when you need to modify items by index:

```
python
nums = [1, 2, 3]
for i in range(len(nums)):
    nums[i] = nums[i] * 10
print(nums)       # [10, 20, 30]
```

Looping through two lists together: zip()

```
python
ports = [22, 80, 443]
names = ["ssh", "http", "https"]


for port, name in zip(ports, names):
    print(f"{port}: {name}")
```

Looping through a list of tuples

```
python
servers = [("web-01", 80), ("ssh-01", 22), ("db-01", 3306)]


for name, port in servers:
    print(f"{name} listens on {port}")
```

Filtering with a loop

```
python
ports = [21, 22, 80, 443, 3306, 8080]
```

```
high_ports = []


for p in ports:
    if p > 1023:
        high_ports.append(p)


print(high_ports)    # [3306, 8080]
```

List comprehension (bonus)

A compact form of the loop above:

python

```
high_ports = [p for p in ports if p > 1023]
squares = [n * n for n in range(1, 6)]    # [1, 4, 9, 16, 25]
```

Use it for simple cases; a normal loop is better when the logic gets complex.

Don't edit a list while looping over it

python

```
nums = [1, 2, 3, 4, 5, 6]


# WRONG: skips items because the list shifts while looping
for n in nums:
    if n % 2 == 0:
        nums.remove(n)


# RIGHT: loop over a copy, or build a new list
nums = [n for n in nums if n % 2 != 0]
```

Removing duplicates (preview)

python

```
ips = ["10.0.0.5", "10.0.0.7", "10.0.0.5"]
unique = []
for ip in ips:
    if ip not in unique:
        unique.append(ip)
```

```
print(unique)    # ['10.0.0.5', '10.0.0.7']
```

(Day 12-ish topics such as sets do this in one line.)

Putting It All Together: Allowed Port Auditor

This compares a **sample list of open ports** (stored in the script) against a fixed policy. Nothing is scanned.

Create day10_program.py:

```
python

# Program: Day 10 lists and tuples

# Purpose: Audit sample open ports against an allowed list (practice only)


ALLOWED_PORTS = (22, 80, 443)              # tuple: policy should not change


open_ports = [22, 80, 3389, 443, 21, 8080]  # sample data from a lab machine


unexpected = []

allowed_found = []


print("=" * 34)

print("  PORT POLICY AUDITOR")

print("=" * 34)


for port in open_ports:

    if port in ALLOWED_PORTS:

        allowed_found.append(port)

    else:

        unexpected.append(port)


unexpected.sort()


print(f"Total open ports:   {len(open_ports)}")

print(f"Allowed ports:      {allowed_found}")

print(f"Unexpected ports:   {unexpected}")
```

```
print()


if unexpected:
    print("Review these ports:")
    for i, port in enumerate(unexpected, start=1):
        print(f"{i}. Port {port}")
else:
    print("All open ports are allowed.")


# Which allowed ports are NOT open?
closed_allowed = [p for p in ALLOWED_PORTS if p not in open_ports]
print()
print(f"Allowed but not open: {closed_allowed}")
```

Output:

```
==================================
  PORT POLICY AUDITOR
==================================
Total open ports:   6
Allowed ports:      [22, 80, 443]
Unexpected ports:   [21, 3389, 8080]


Review these ports:
1. Port 21
2. Port 3389
3. Port 8080


Allowed but not open: []
```

This is the idea behind real configuration audits: compare what *is* running with what *should* be running, and flag the difference.

Bonus: an interactive username list manager (practice only):

```
python
users = ["alice", "bob"]
```

```
while True:
    cmd = input("Command (add/remove/list/quit): ").strip().lower()

    if cmd == "quit":
        break
    elif cmd == "list":
        print(users if users else "No users")
    elif cmd == "add":
        name = input("Name: ").strip().lower()
        if name == "":
            print("Name cannot be empty")
        elif name in users:
            print("Already exists")
        else:
            users.append(name)
            print(f"Added {name}")
    elif cmd == "remove":
        name = input("Name: ").strip().lower()
        if name in users:
            users.remove(name)
            print(f"Removed {name}")
        else:
            print("Not found")
    else:
        print("Unknown command")
```

Practice Tasks

1. Create a list of five lab hostnames. Print the first, last, and middle items.
2. Print the first three and last two hostnames using slicing, then print the list reversed.
3. Start with ports = [22, 80]. Append 443, insert 21 at the start, and extend with [25, 53]. Print after each step.

4. Remove port 25 safely (only if it exists), then pop the last item and print it.
5. Sort [443, 22, 3389, 80, 21] ascending and descending, once with sort() and once with sorted(). Confirm which one changes the original.
6. Create a tuple ("192.168.56.101", 22), unpack it into host and port, and print them with an f-string. Then try to change host[0] and read the error.
7. Prove the copy trap: create a, assign b = a, change b, and print both. Fix it with .copy().
8. Given ports = [21, 22, 23, 80, 443, 3389], build a new list of only the ports below 1024 using a loop, then again with a list comprehension.
9. Given scores = [72, 85, 90, 64], print the highest, lowest, sum, and average (sum / len) formatted to 1 decimal place.
10. Use zip() to pair ["ssh", "http", "https"] with [22, 80, 443] and print one line per pair.
11. Ask the user for a comma-separated list of ports (for example 22,80,443), split it, keep only items that pass isdigit(), convert them to integers, and print the sorted result.
12. Extend the Port Auditor: add a RISKY_PORTS = (21, 23, 3389) tuple, and print a separate warning list of any open ports that appear in it.
13. Predict the output, then test:

python

```
a = [1, 2, 3]
b = a[:]
b.append(4)
c = a
c.append(5)
print(a, b, c)
```

Want me to continue with Day 11 (dictionaries), or compile Days 1 to 10 into a Word document or slide deck?